ME5418 - Learning Agent C&R

Team 41: Benjamin Teh Jhen Hing, Lu Jia Yih Jensen, Wee Fook Choon

1 Code Explanation

1.1 Code Organization & Existing Code/Libraries

The learning agents connect our Gymnasium environment (52-dim state, `Discrete(36)` actions) to a neural network and turn interaction data into gradient updates. In `src/learning_agents/` we implemented an off-policy value-based agent, `DuelingDDQNAgent` (`dueling_ddqn_agent.py`, with `ReplayMemory` in `experience_replay.py` and the Assignment 2 `DQN` network), and an on-policy policy-gradient agent, `PPOAgent` (`ppo_agent.py`, with a new `ActorCritic` network in `neural_nets/ppo_torch.py` using the same trunk and head widths, so both have 150,949 parameters). Hyperparameters live in `DQNConfig`/`PPOConfig` (`config/nn_config.py`). Both agent classes share one layout: `run(is_training)` contains the episode loop for training *and* evaluation (evaluation loads the checkpoint and acts greedily); `optimize()` performs the gradient update; `_step_with_action_repeat()` wraps `env.step()`; and `save_graph()` plots the 100-episode mean reward with the exploration signal (ϵ or policy entropy). PPO adds `_compute_gae()`. Weights are saved whenever the 100-episode mean reward improves.

Existing code/libraries:

- **Python Libraries** : `PyTorch` (`Adam`, `SmoothL1Loss`, `F.mse_loss`, `Categorical`, `clip_grad_norm_`), Gymnasium,NumPy, Matplotlib.
- **The DQN agent** loop is adapted from johnnycode8's PyTorch DQN tutorial (`https://github.com/johnnycode8/dqn_pytorch`); we replaced its MSE loss with Huber loss and gradient clipping, its once-per-episode update with an update every 4 steps after a warm-up, its single-best-episode checkpoint with a 100-episode mean, and added truncation handling and action repeat.
- **The PPO** update follows CleanRL's `ppo.py` (`https://github.com/vwxyzjn/cleanrl`) implementation details (GAE, clipped loss, minibatch advantage normalization, `Adam(eps=1e-5)`, clipping at 0.5), rewritten into the same single-environment agent structure as our DQN agent.

1.2 Shared Interaction Wrapper: Action Repeat & Jump Release

Each action is held for 4 frames (15 Hz decisions instead of 60 Hz) and the 4 rewards are summed into one transition. Because jumping is edge-triggered (`Car._apply_jump` reacts only to a released→pressed change), the wrapper clears the jump bit on repeat frames 1–3; otherwise double-jumps and dodges would be impossible.

1.3 Off-Policy Dueling Double DQN Agent

The agent learns $Q_\theta(s, a)$ and acts ϵ-greedily (see Algorithm 1). Each component addresses a known failure mode of naive Q-learning with neural networks:

- **Experience replay** ($N = 10^5$, uniform minibatches of 64 after a 2,000-step warm-up, one update every 4 steps) breaks the strong correlation between consecutive frames and reuses each transition many times.
- **Target network** Q_{θ^-} (copied every 1,000 steps) keeps the regression target fixed between syncs, avoiding a "moving target" feedback loop.
- **Double DQN target** $y = R + \gamma(1 - \texttt{terminated})\, Q_{\theta^-}(s', \arg\max_{a'} Q_\theta(s', a'))$: selecting with the online net and evaluating with the target net reduces the over-estimation caused by max over noisy estimates.
- **Huber loss** with gradient-norm clipping at 10 limits the effect of large TD errors from ±20 goal rewards; ϵ decays per episode as $\max(0.05,\, 0.9995^k)$.

Algorithm 1: Dueling Double DQN agent (`DuelingDDQNAgent.run`, `optimize`)

Input: Online net Q_θ, target net $Q_{\theta^-} \leftarrow Q_\theta$, replay buffer $\mathcal{D}$, $\gamma = 0.99$, $\epsilon \leftarrow 1.0$

for *episode* $k = 1, 2, \ldots$ **do**
  $s \leftarrow$ env.reset()
  while *not* (*term* $\vee$ *trunc*) **do**
    $a \leftarrow$ random action w.p. ϵ, else $\arg\max_{a'} Q_\theta(s, a')$
    $s', R,$ `term`, `trunc` $\leftarrow$ `_step_with_action_repeat`(env, a); store $(s, a, s', R,$ `term`$)$ in $\mathcal{D}$; $s \leftarrow s'$
    if $|\mathcal{D}| \geq 2{,}000$ ***and*** *step* $\bmod 4 = 0$ **then**
      Sample 64 transitions; $y_i \leftarrow R_i + \gamma(1 - \texttt{term}_i)\, Q_{\theta^-}(s_i', \arg\max_{a'} Q_\theta(s_i', a'))$
      Adam step on Huber$(Q_\theta(s_i, a_i), y_i)$ with $\|\nabla\|_2 \leq 10$
    if *step* $\bmod 1{,}000 = 0$ **then** $\theta^- \leftarrow \theta$
  $\epsilon \leftarrow \max(0.05, 0.9995\,\epsilon)$; save weights if 100-episode mean reward improves

1.4 On-Policy PPO Actor-Critic Agent

PPO learns a stochastic policy $\pi_\theta(a|s)$ (actor) and a baseline $V_\phi(s)$ (critic) directly, so it explores by sampling rather than ϵ (see Algorithm 2):

- **Rollout & GAE:** $T = 4{,}096$ steps are collected with the current policy, then `_compute_gae` sweeps backwards to compute advantages $\hat{A}_t$ ($\gamma = 0.99$, $\lambda = 0.95$). λ trades the low variance of 1-step TD against the low bias of Monte-Carlo returns. Returns $\hat{G}_t = \hat{A}_t + V_\phi(s_t)$ train the critic.
- **Clipped surrogate:** the data is reused for 6 epochs of shuffled 256-sample minibatches. The probability ratio $r_t = \pi_\theta / \pi_{\theta_{\text{old}}}$ is clipped to $[0.8, 1.2]$ so one rollout cannot push the policy too far. A $0.5\times$ critic MSE term and a 0.01 entropy bonus (to keep exploring) are added to the loss.

Algorithm 2: PPO agent (`PPOAgent.run`, `_compute_gae`, `optimize`)

Input: `ActorCritic` (π_θ, V_ϕ), rollout length $T = 4{,}096$, $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.2$

while *training* **do**
  for $t = 1, \ldots, T$ **do**
    $a_t \sim \pi_\theta(\cdot|s_t)$; record $\log \pi_{\theta_{\text{old}}}(a_t|s_t)$, $V_\phi(s_t)$
    $s_{t+1}, R_t, \texttt{term}_t, \texttt{trunc}_t \leftarrow$ `_step_with_action_repeat`(env, a_t); $d_t \leftarrow \texttt{term}_t \vee \texttt{trunc}_t$
    $V_t^{\text{next}} \leftarrow 0$ if $\texttt{term}_t$ else $V_\phi(s_{t+1})$ `// timeouts still bootstrap`
  for $t = T, \ldots, 1$ **do**
    $\delta_t = R_t + \gamma V_t^{\text{next}} - V_\phi(s_t)$; $\quad \hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1}$; $\quad \hat{G}_t = \hat{A}_t + V_\phi(s_t)$
  for 6 *epochs, each shuffled minibatch of* 256 **do**
    Normalize $\hat{A}$ batch; $r_t = \exp(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t))$
    Adam step on $-\min(r_t\hat{A}_t, \text{clip}(r_t, 1 \pm \epsilon)\hat{A}_t) + 0.5\,(V_\phi(s_t) - \hat{G}_t)^2 - 0.01\,\mathcal{H}(\pi_\theta)$ with $\|\nabla\|_2 \le 0.5$
  Clear rollout; save weights if 100-episode mean reward improves

2 Reflections & Lessons Learned

- **Changes from proposal:** The proposal planned PPO; Assignment 2 switched to Dueling DDQN. We now implement both with matched networks, learning rate, γ and action repeat, so differences come from the algorithm rather than model capacity. As a simplification, PPO also uses the flattened 36-action output instead of separate direction/jump/boost heads. Since our Assignment 2 single unbatched update at $\text{lr} = 10^{-3}$ overshot its target, we use minibatches of 256 and $\text{lr} = 3 \times 10^{-4}$.
- **Correlated data vs. sequential credit:** Consecutive frames are highly correlated, which biases SGD. Both agents first capture the time order (DQN stores 1-step transitions; PPO computes $\hat{A}_t$ over the ordered rollout) and only then shuffle samples into minibatches.
- **Horizon & action timing:** At 60 Hz a 3,000-frame episode makes distant goals almost invisible ($\gamma^{3000} \approx 0$) and the policy chatters between actions. Action repeat cut the horizon to ≤ 750 decisions. Action repeat also interacts with game mechanics: because jumping is edge-triggered, a held jump bit would disable double-jumps and dodges, so the agent wrapper must release it.
- **Terminated vs. truncated:** Treating 3,000-step timeouts like goals would teach the agent that the state just before a timeout is worth 0. Only true goals zero the bootstrap: DQN masks with `terminated` alone, and PPO bootstraps $V_\phi(s_{t+1})$ on timeouts.
- **Reward hacking & reward scale:** With +20 for every Blue goal, the agent learned to bait the heuristic bot into own-goals instead of attacking; own-goals by the opponent now give only +2. Goal rewards (± 20) also dwarf shaping terms, so DQN needs Huber loss, and PPO relies on advantage normalization, ratio clipping and tighter gradient clipping.
- **Batch Normalisation:** Batch normalisation is performed on the entire batch of 4,096 samples before minibatches are sampled because goals are rare occurrences that may be omitted from the minibatches. Thus normalising across the batch maintains the scale of the advantages.
- **Remaining changes:** In preliminary runs Dueling DDQN reached a 100% goal rate against the heuristic bot but PPO only 56% after ∼6,000 episodes, consistent with PPO discarding each rollout after 6 epochs while DQN reuses transitions. For the final project we plan to tune PPO, use a curriculum (passive bot → HZISM bot → mirrored self-play), and compare the agents head-to-head.

AI Tool Declaration: We used Google Gemini and Claude Code to write and debug code, format paragraphs, and condense the LaTeX layout. We are responsible for the content and quality of the submitted work.